

Homework 3

Make a CPU

Your task is to build an 8-bit CPU / computer in logisim, which runs an instruction set of your design. Your instruction set should implement (at least) the following instructions:

- AND
- OR
- add
- addi (easiest to do first!)
- sub
- slt
- NOR
- beq (best to save for last)
- bne (best to save for last)
- j
- lw
- sw
- no-op

This instruction set is very short (compared to x86 for example, which has hundreds of instructions). It is also very inefficient for most things, but it can be considered a “complete” CPU. You are free to use any of the things we’ve built in class, but there are several Logisim items you should avoid using (e.g., the divider). In short, you should be able to build the entire computer using only the components listed below. If you find that you want to use something not on this list, please feel free to ask me about it.

- Register
- ALU
- Memory
 - RAM for data
 - ROM for instructions
- Clock
- Splitter
- Bit Extender (set to sign-extend)
- Multiplexers / Demultiplexer
- Button & LED
- 7 Logical Gates (AND, OR, NOT, NAND, NOR, XOR, XNOR)
- Anything from the “wiring” folder of components in Logisim

Grading

Your computer will be graded using this program.

Hash-tag (pound-sign) indicates a comment

```
0 no-op          # do nothing
```

```
1 addi $r1 $r2 3  # r1:3
```

```

2 addi $r2 $r0 2    # r2:2

3 add $r3 $r1 $r2    # r1:3  r2:2  r3:5

4 sub $r4 $r2 $r1

5 and $r5 $r1 $r2

6 or $r6 $r1 $r2

7 slt $r7 $r2 $r3

8 nor $r3 $r2 $r3

9 sw $r3 1($r1)      # mem[3+1]:0xf8

10 lw $r4 2($r2)     # r4: 0xf8

11 bne $r2 $r3 13    # goto instruction 13

12 addi $r1, $r0, 0x0D # should be skipped!

13 beq $r2, $r5 15

14 addi $r2, $r0, 0x0E

15 j 1                # goto line 1

```

You will need to translate the above program from my pseudo-assembly, to machine language for your computer / cpu / ISA. Your grade will be based primarily on how many of the instructions function correctly in your computer. Specifically, each successive instruction that your computer executes will be worth about one point out of thirteen.

Road Map

We've built a few simple machines in class. The computer will be like a more versatile version of those machines. Over the next two weeks, I will give lectures that will help explain how to go about this process. I highly recommend that you start working early, so that you can bring questions to class. I will have class time every day until the project is finished devoted to answering questions in detail. Here is a high-level outline you should take in building your computer in logisim.

1. Build and test a working 8-bit ALU.

2. Build and test a working register file.
3. Build a program counter and use its value to address the program memory.
4. Design your instruction set (loosely) on paper or using a spreadsheet program.
5. Split the output of each instruction into components based on your instruction format in logisim.
6. Pick one instruction and wire it up completely to an ALU, register file, and the main memory (I recommend starting with r-type instructions).
 - Make sure the “Asynchronous read” property is set to “Yes” on the RAM
 - Make sure the “Trigger” property is set to “Falling Edge” on the RAM
7. Add instructions by implementing control lines (in (a) control module(s)).
8. Test the machine as you add each instruction. Write a simple program and make sure it does what you think it should.
9. Translate and code the grading program into your computer ROM in machine code, test it.
10. Fix anything you find that’s wrong!
11. Submit.

Submitting

Please submit to me (a) the logisim file containing your CPU project (and all necessary circuits) and (b) the binary representation of the given program. Please name your computer creatively, for example I called mine `Holly.circ`